AI COURSE – II AIML / REC

CATEGORY 7 – CONSTRAINT SATISFACTION PROBLEMS

Combined Menu-Driven Python Program

This program combines Map-Coloring using backtracking, the SEND + MORE = MONEY crypt-arithmetic puzzle, and Local Search using the Min-Conflicts method for the 8-Queens problem.

Program

```
import random
from itertools import permutations


# 1. MAP-COLORING USING BACKTRACKING

def map_coloring(graph, colors, result):

    if len(result) == len(graph):
        return True

    region = next(r for r in graph if r not in result)

    for color in colors:
        safe = True

        for neighbor in graph[region]:
            if result.get(neighbor) == color:
                safe = False
                break

        if safe:
            result[region] = color

            if map_coloring(graph, colors, result):
                return True

            del result[region]

    return False


def solve_map_coloring():

    graph = {
        'WA': ['NT', 'SA'],
        'NT': ['WA', 'SA', 'Q'],
        'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
        'Q': ['NT', 'SA', 'NSW'],
        'NSW': ['SA', 'Q', 'V'],
        'V': ['SA', 'NSW'],
        'T': []
    }

    colors = ['Red', 'Green', 'Blue']
    result = {}

    if map_coloring(graph, colors, result):
        print("Map Coloring Solution:")
        for region, color in result.items():
            print(region, ":", color)
    else:
        print("No Solution")


# 2. CRYPT-ARITHMETIC: SEND + MORE = MONEY

def solve_crypt_arithmetic():

    letters = 'SENDMORY'

    for p in permutations(range(10), 8):

        d = dict(zip(letters, p))

        if d['S'] == 0 or d['M'] == 0:
            continue

        send = (d['S'] * 1000 + d['E'] * 100 +
                d['N'] * 10 + d['D'])

        more = (d['M'] * 1000 + d['O'] * 100 +
                d['R'] * 10 + d['E'])

        money = (d['M'] * 10000 + d['O'] * 1000 +
                 d['N'] * 100 + d['E'] * 10 + d['Y'])

        if send + more == money:
```

```
            print("Crypt-Arithmetic Solution:")
            print("SEND =", send)
            print("MORE =", more)
            print("MONEY =", money)
            return

    print("No Solution")


# 3. LOCAL SEARCH USING MIN-CONFLICTS (8-QUEENS)

def conflicts(board, col, row):

    count = 0
    n = len(board)

    for c in range(n):
        if c != col:
            r = board[c]

            if r == row or abs(r - row) == abs(c - col):
                count += 1

    return count


def solve_queens():

    n = 8
    random.seed(7)

    for restart in range(100):

        board = [random.randrange(n) for _ in range(n)]

        for step in range(10000):

            bad = [c for c in range(n)
                   if conflicts(board, c, board[c]) > 0]

            if not bad:
                print("8-Queens Solution:")
                for r in range(n):
                    for c in range(n):
                        if board[c] == r:
                            print("Q", end=" ")
                        else:
                            print(".", end=" ")
                    print()
                return

            col = random.choice(bad)
            scores = [conflicts(board, col, r) for r in range(n)]
            minimum = min(scores)
            best_rows = [r for r in range(n) if scores[r] == minimum]
            board[col] = random.choice(best_rows)

    print("No Solution Found")


# MAIN MENU

print("CONSTRAINT SATISFACTION PROBLEMS")
print("1. Map-Coloring")
print("2. Crypt-Arithmetic")
print("3. Local Search (8-Queens)")

choice = int(input("Enter your choice: "))

if choice == 1:
    solve_map_coloring()

elif choice == 2:
    solve_crypt_arithmetic()

elif choice == 3:
    solve_queens()

else:
    print("Invalid Choice")
```

Sample Outputs

Map-Coloring

```
Map Coloring Solution:
WA : Red
NT : Green
SA : Blue
Q : Red
NSW : Green
V : Red
T : Red
```

Crypt-Arithmetic

```
Crypt-Arithmetic Solution:
SEND = 9567
MORE = 1085
MONEY = 10652
```

Local Search (8-Queens)

```
8-Queens Solution:
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . . . . . Q
. . . . Q . . .
Q . . . . . . .
. . . Q . . . .
. . . . . Q . .
```

Result

Thus, a combined menu-driven Python program for Map-Coloring, Crypt-Arithmetic, and Local Search for CSP was implemented successfully.

Note: The Local Search section uses random restarts with Min-Conflicts to improve the chance of finding an 8-Queens solution.